Date : ☐M ☐T ☐W ☐T ☐F ☐S ☐S

* What is maven :→

Maven is a build automation and project management tool primarily used for Java projects.

It simplifies the process of building and managing Java-based applications (by handling.

It is used for project build, dependency and documentation based on POM (project object model).

Maven is used to →

① Project Build :→

Maven automates the compilation of code, packaging it into .jar or .war files, running tests, and deploying the project.

② Dependency Management :→

You can define all require libraries (dependencies) in single XML file (pom.xml) and Maven will automatically download them from central repositories.

Example :→

xml

```
<dependencies>

    <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version> 3.00 </version>

    <dependency>

</dependencies>
```

③ Standard Project Structure :→

Maven automatically creates an standard directory layout.

```
project-root/
├── src/
│   └── main/
│       ├── java/
│       └── resources/
│   └── test/
├── target/
└── pom.xml
```

Date : ☐M ☐T ☐W ☐T ☐F ☐S ☐S

④ POM (Project Object Model) :→

The pom.xml file is the core of every Maven project.

It defines → i) Project info (name, version)

ii) Dependencies.

iii) Build Plugins.

iv) Configuration.

**

* Problems when we not use maven and create project without using Maven →

i) Need to create the right project structure.

ii) Need to add set of Jars in each project. download dependencies manually

iii) Building and Deploying the project.

(Pagination, Caching)

Date:

* What is Hibernate :→

Hibernate is an ORM (Object Relational Mapping) framework for java that allows developers to map Java objects (classes) to relational database tables without writing complex SQL queries.

Why ORM :→

Normally, Java applications use JDBC to connect to a database, which involves :→

i) Writing SQL manually.

ii) Managing result set.

iii) Handling connections.

iv) Performing conversions between Java and SQL data types.

{ ORM solves this by other approaches that is next }

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

ORM solves this by →

i) Automatically generating SQL queries.

ii) Handling Java to SQL data conversion.

iii) Managing relationship (e.g OneToMany, ManyToMany).

iv) Providing caching and transaction Management.

Benefits of Hibernate →

i) Simplified Code - No need to write SQL manually for CRUD.

ii) Portable - works with almost all SQL databases.

iii) HQL (Hibernate Query Language) - Similar to SQL but works on entity objects instead of table names.

iv) Automatic Table Creation - Automatically create/updates tables using annotations and configuration.

{ Hibernate supports two type of mapping
i.e i) Annotation - Based Mapping
ii) XML - Based Mapping }

① Annotation - Based Mapping →

i) Uses Java annotations to define mappings directly in your entity classes.

ii) Cleaner and more morden approach.

iii) Follows JPA (Java persistence API) specification.

Example :→

```
@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.Auto
    private int id;

    @Column(name = "student_name")
    private String name;

    // Constructor and setter/getter.
}
```

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

② XML - Based Mapping →

- Uses external .hbm.xml files to define mappings.
- Was the old technique used before annotations were introduced.

Example (Student.hbm.xml) →

```
<hibernate-mapping>
    <class name = "Student" table = "student">
        <id name = "id" column = "id">
            <generator class = "increment" />
        </id>
        <property name = "name" column = "stu_name" />
    </class>
</hibernate-mapping>
```

{ The configuration in both differs slightly in hibernate.cfg.xml

① In Annotation-based Mapping (hibernate.cfg.xml)

```
<mapping class = "com.example.Student" />
```
(fully qualified name)

② In XML-based Mapping (hibernate.cfg.xml)

```
<mapping resources = "Student.hbm.xml" />
```
(file name)

* List of all important Hiberate annotations.

① @Entity :→

i) Marks a class as a Hiberbate entity (table in the database).

ii) This tells Hibernate: "This class should be stored in the database".

② @Id :→

i) Marks a field as the primary key.

ii) Used to uniquely identify each record.

③ @GeneratedValue :→

i) Automatically generates the value of the primary key.

ii) You don't need to manually assign ID values.

4 Types → i) GenerationType.AUTO
ii) GenerationType.IDENTITY
iii) GenerationType.SEQUENCE.
iv) GenerationType.TABLE.

(Most preferrable is AUTO).

Date : ☐M ☐T ☐W ☐T ☐F ☐S ☐S

④ @Column

i) Used to customize a column's name or properties in the table.

ii) You can skip it if you want Hibernate to use field name directly (i.e variable name)

⑤ @OneToOne

i) One entity is associated with exactly one other entity.

ii) One person → one passport

```
@OneToOne
private Passport passport;
```

⑥ @OneToMany

i) One entity is related to many other entities

ii) One department → many employees.

```
@OneToMany(mappedBy = "department",
           cascade = CascadeType.ALL)
private List<Employee> employee;
```

⑦ @ManyToOne

i) The reverse of @OneToMany, used on the child side.

ii) Many employees → one department.

```
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

⑧ @ManyToMany

i) Many entities can be related to many other entities.

ii) Many Students ⟷ Many courses

```
@ManyToMany
@JoinTable(
    name = "student_course",
    joinColumn = @JoinColumn(name = "stu_id"),
    inverseJoinColumns = @JoinColumn(
                            name = "course_id")
)
private List<Course> course;
```

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

⑨ @JoinColumn

i) Specifies the foreign key column in a table.

ii) Tells Hibernate which column connects two tables.

@JoinColumn (name = "dept_id")

⑩ @JoinTable

i) Defines a join table for @ManyToMany relationship.

ii) It connects the primary keys of two tables.

⑪ @MappedBy

i) Used in bidirectional relationships to specify the owner of the mapping

ii) Avoids redundant Mapping.

@OneToMany (mappedBy = "department").

(12) Cascade = CascadeType.ALL

- Allows Hibernate to automatically apply changes (save, delete, etc) to related entities.

```
@OneToMany(cascade = CascadeType.ALL)
```

(13) @Embeddable

- Used to define a class that can be embedded into another entity.

```
@Embeddable
public class Address
{
.....
}
```

(14) @Embedded

- Used to embed an @Embeddable class into an entity.

- No new table is created; fields go into the same table as the parent entity.

```
@Embedded
private Address address;
```

Date : ☐M ☐T ☐W ☐T ☐F ☐S ☐S

* Hibernate Object States and Lifecycle

What are Hibernate Object States →

In Hibernate, an object (i.e, an entity) goes through different states based on whether it is associated with a Hibernate session and whether its data is synchronized with the database.

The (3) Main object States →

1. Transient
2. Persistent
3. Detached.

(1) Transient State :→

i) The object is created using the new keyword

ii) It is not associated with any Hibernate session.

iii) It is not stored in the database.

iv) No primary key assigned here.

v) Not tracked by Hibernate.

```
Student s = new Student();    // Transient.
s.setName("Ankit");
```

② Persistent State :→

i) The object is associated with a session

ii) Any changes made to the object are automatically synchronized with the database when the session is flushed or closed.

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
session.save(s);    // s is now Persistent.
s.setName("Aniket");    // Auto updated in DB.
tx.commit();
session.close();
```

iii) Hibernate track it

iv) Automatic update on commit().

Date : ☐M ☐T ☐W ☐T ☐F ☐S ☐S

③ Detached State :→

i) The object was persistent, but the session is now closed

ii) The object still holds a database identity (ID), but is no longer managed by Hibernate.

```
session.close();          // s becomes Detached.

s.setName("Updated Name"); // No auto-update to DB.
```

** Example Code of full Lifecycle :→

```
Session session = factory.openSession();

Transaction tx = session.beginTransaction();

Student s = new Student();   // Transient
s.setName("Ankit");

session.save(s)              // Persistent
s.setName("Ankit Thorat");

tx.commit();
session.close();             // Detached.
```

(difference between get() & load())

☐M ☐T ☐W ☐T ☐F ☐S ☐S Date:

* Hibernate Architecture :⇒

① Configuration :⇒

i) Loads Hibernate settings from hibernate.cfg.xml or hibernate.properties.

ii) Sets database connection, dialect, mappings, etc

```
Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");
```

② SessionFactory :⇒

i) Heavy-weight object. Create once per app.

ii) Contains pooled DB connections.

iii) Created from Configuration.

```
SessionFactory factory = cfg.buildSessionFactory();
```

Date: DM DT DW DT DF DS DS

③ Session :→

i) Represents a single unit of work with the DB.

ii) Handles saving, retrieving, updating, & deleting objects.

iii) It is not thread-safe.

```
Session session = factory.openSession();
```

④ Transaction :→

i) Used to group database operations into a single atomic unit.

ii) Ensure data consistency.

```
Transaction tx = session.beginTransaction();
tx.commit();
```

M T W T F S S

Date :

* Hibernate Configuration →

hibernate.connection.driver_class → JDBC Driver.

hibernate.connection.url → DB connection url

hibernate.connection.username /
hibernate.connection.password → DB credentials

hibernate.dialect → SQL dialect for the DB.

hibernate.hbmddl.auto → Auto schema management (create, update, validate, none).

show_sql → logs the SQL queries to console.

Entity mapping →

2 approaches

i) Annotation based

ii) XML based.

Date: DM DT DW DT DF DS DS

Example → (hibernate.cfg.xml)

```
<hibernate-configuration>
  <session-factory>
    <property name="connection.driver_class">
      com.mysql.cj.jdbc.Driver </property>
    <property name="connection.url">
      jdbc:mysql://localhost:3306/mydb
      ?createDatabaseIfNotExist=true
      </property>
    <property name="connection.username">
      root </property>
    <property name="connection.password">
      Ankit2631 </property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQLDialect
      </property>
    <property name="hibernate.hbm2ddl.auto">
      update </property>
    <property name="show_sql">true
      </property>

    <!-- Entity Mapping -->

    <mapping class="com.model.Student" />

  </session-factory>
</hibernate-configuration>
```

Date:

Example → (Entity class: Student.java)

```
@Entity
@Table(name = "Student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.
                                    AUTO)
    private int id;

    @Column(name = "Name", length = 10)
    private String name;

    @Column(name = "Email")
    private String email;

    @Column(name = "Age")
    private int age;

    // Constructor (default & para)

    // getters & setters

}
```

* Diff bet' get() & load

Date: (IMP) ☐M ☐T ☐W ☐T ☐F ☐S ☐S

* get() Method in Hibernate →

The get() method is used to fetch a persistent entity object from the database based on its primary key.

```
Student student = session.get(Student.class, 1);
// Fetch student with id = 1
```

* What is JPA :→

JPA stands for Java Persistence API. It is a Java specification (like a set of rules) used for storing, retrieving, and managing data in databases using Java objects.

{ JPA allows you to work with a database using Java classes instead of writing complex SQL queries. i.e (Entity) Java class. }

JPA is a standard to manage Java objects and databases without writing SQL.

You define classes (@Entity) instead of SQL tabl

You need to an implementation like Hibernate to actually make it work.